

Universitatea de Stat din Moldova

Facultatea de Fizică și Inginerie

**Infrastructură de măsurare pentru
compararea configurațiilor LLM într-un
pipeline multi-agent**

Penzari Marc
TI231

2026

Context și problemă

- **Revânzarea în e-commerce:** extragere -> rescriere -> imagini -> preț -> publicare;
- **LLM-urile** fac automatizarea robustă: citesc pagina ca un om, produc ieșire structurată;
- **Întrebarea principală:** dintre numeroasele LLM-uri (preț, prompt, implementare diferite), cum se compară pe o sarcină fixă și reproductibilă?
- **Două livrabile:** un flux multi-agent funcțional + o infrastructură de măsurare în jurul lui.

2

- Revânzarea e obositoare și ea mult timp. Etapele enumerate sunt manuale și dau mult timp;
- Spre deosebire de extractori rigizi bazați pe script-uri, ...;
- Întrebarea principală a proiectului: ...;
- ...;

Fundamente tehnice

- **Sistem multi-agent:** sarcina spartă în sub-sarcini, fiecare = un agent (un apel LLM cu prompt și contract de I/O);
- **Renunțare la *framework-uri*:** LangChain / LangGraph / AutoGen -> orchestrator propriu de ~200 linii;
- **Extragere structurată:** mod JSON + validare de schemă (Pydantic) cu reîncercare-o-dată la eșec;
- **Web scraping anti-bot:** impersonare TLS (curl_cffi), rezervă Playwright, recuperare manuală folosind om-în-bucă;
- **Interfață asincronă:** bot Telegram (notificări push pentru rulări lungi).

Idei pe care este bazat proiectul:

- Sarcina este spartă în etape realizate de agenți;
- Framework scris manual;
- Extragerea datelor cu validarea tipurilor;
- Traversarea paginilor cu atenuarea boților;

Arhitectura sistemului

- **Patru servicii care cooperează:** bot, orchestrator, worker (Python 3.12) + Redis ca broker;
- **Separare pentru izolarea eșecurilor:** un apel LLM lent în worker nu blochează botul;
- **Orchestrator (FastAPI):** API REST, ciclul de viață al rulării. Worker (**arg**): execută fluxul, înregistrează măsurători;
- Stocare:
 - **runs:** o linie per rulare (mașină de stări: queued -> running -> awaiting_approval -> published/...);
 - **step_runs:** o linie per apel agent (model, latență, token-uri, cost, eroare);
 - Artefacte JSON/imagini pe disc, per rulare.

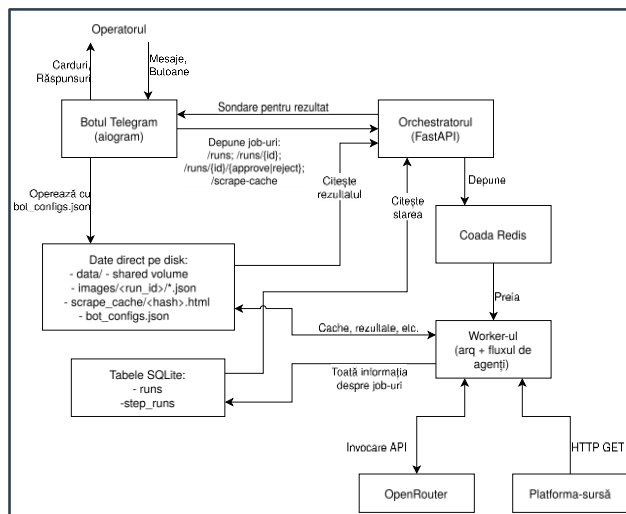
4

Din ce este compus proiectul:

- Patru servicii principale:
 - bot telegram;
 - orchestrator: managerul sistemului, interfața de cooperare user-worker;
 - worker: serviciul care efectuează lucrul principal;
 - Redis: sistemul de queue;
- Separarea pe servicii permite rulare independentă și izolarea eșecurilor;
- Orchestratorul este bazat pe FastAPI, observă ciclul de viață; Worker-ul execută job-ul;
- Datele stocate pe disc: ...;

Pipeline-ul

- Sase agenți secvențiali (3 folosesc LLM):
 - *Scraper*;
 - Extractor;
 - Selector imagini;
 - Rescriitor;
 - Evaluator preț;
 - Compozitor calitate;



5

Din ce este compus pipeline-ul:

- Worker-ul:
 - Rulează job-urile, salvează rezultatele pe disc în: SQLite + fișiere pe disc (rezultatatele OpenRouter, site cache);
 - Job-urile sunt executate secvențial de 6 agenți. Agenții LLM sunt subliniați;
- Botul telegram:
 - Operează cu orchestratorul prin HTTP API + își salvează datele sale pe disc – *bot_configs.json* (configurațiile posibile + cele active);
- Orchestratorul:
 - Expune HTTP API CRUD (Create Read Update Delete) a job-urilor, numai că fără Delete;
 - Botul telegram pune job-urile în queue prin API, apoi le sondează și așteaptă rezultatul prin API;

- Joburile noi sunt puse în Redis queue, de unde le preia worker-ul;

Configurarea experimentului

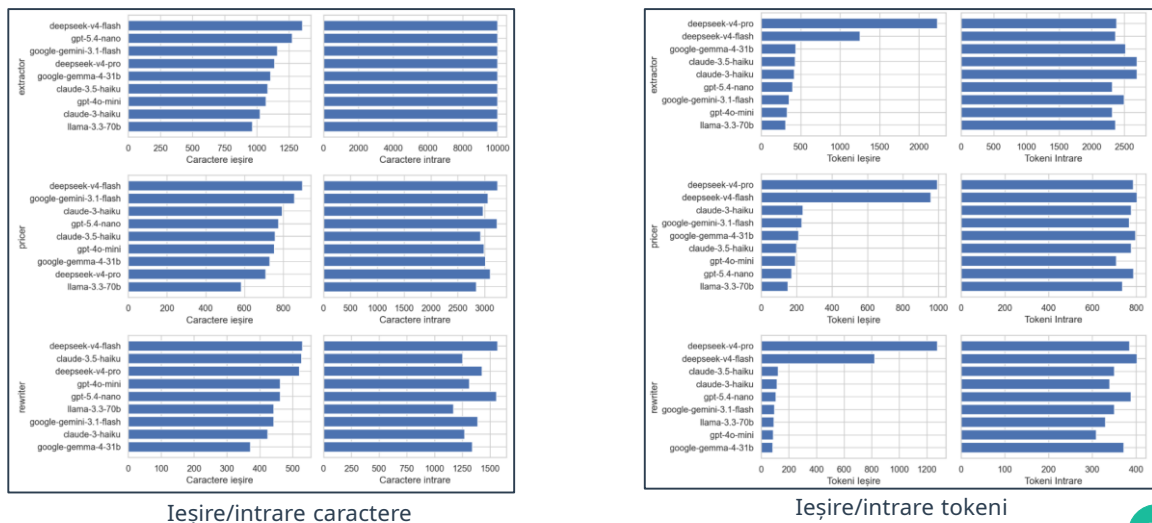
- **9 configurații:** Claude, DeepSeek, GPT, Gemini, Gemma, Llama;
- Matrice de **189 de rulări** = 9 configurații × 7 anunțuri eBay × 3 repetări.
- **HTML din cache:** fiecare URL descărcat o singură dată;
- **Persistență la nivel de apel:** fiecare apel LLM salvat pe disc.

6

Cum au fost rulate experimentele:

- 9 configurații: 2xClaude, 2xDeepSeek, 2xGPT, Gemini, Gemma, Llama;
- Rulări: 9 configurații, 7 link-uri, 3 repetări per link;

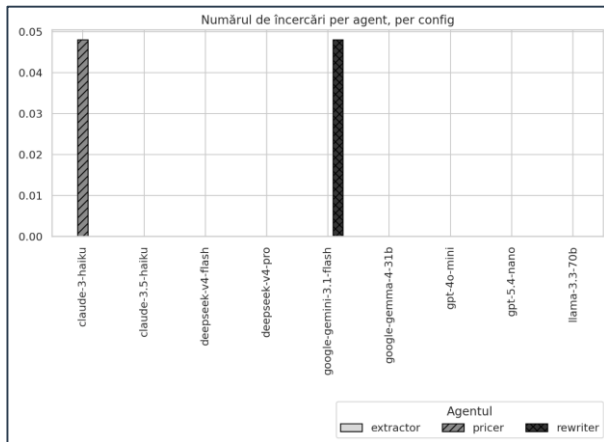
Rezultate cantitative



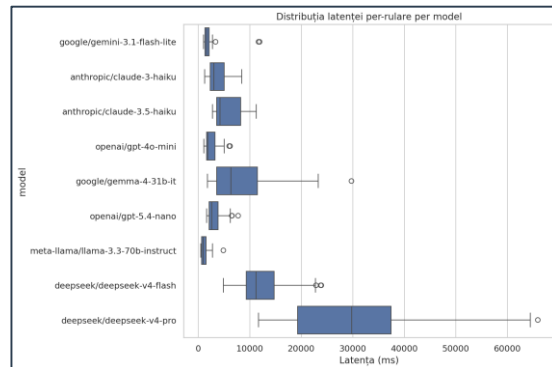
7

- Cantitatea ieșirii/intrării prompt-urilor sunt calculate în token-uri;
- 1 token != 1 caracter;
- Numărul de caractere ieșire/intrare variază cu 44%;
- Numărul de token-uri variază cu 1200%;
- Cauza: tokenizator. Tokenizatorul transformă cuvintele în numere cu care rulează LLM-urile, apoi înapoi în cuvinte;
- Rezultat: Fiecare LLM interpretează cuvintele în felul său -> Unele LLM-uri operează cu cantități diferite de token-uri;

Latența și eșecuri



Numărul de încercări



Distribuția latenței

Trei tipuri de eșecuri:

- Eșuat-apoi-reușit: ex.: apelul LLM a eșuat prima dată, a reușit a doua dată (dacă un apel LLM eșuază prima dată, al doilea apel include și eroarea eșecului);
- Eșec dur: ambele apeluri au eșuat;
- Eșec moale: specific agentului `pricer`. Ambele eșecuri LLM nu generează o excepție, dar marchează că au fost eșuate;

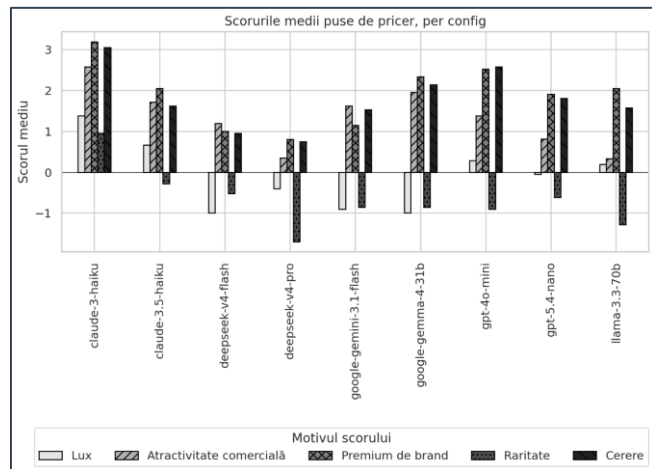
S-au observat 0 eșecuri dure și moale, 2 reîncercări. Rezultat: LLM-urile moderne se adveră foarte bine disciplinei JSON;

Latența:

- În acest caz, latența este măsurată și pentru modelele LLM, și pentru provideri diferiți;

Scoruri prețului

- **Dispoziția de preț:** scoruri pe 5 dimensiuni (-5..+5):
 - Lux;
 - Atractivitatea comercială;
 - Brand;
 - Raritate;
 - Cerere pe piață;



9

- Agentul `pricer` crează un preț nou pentru produs;
- Prețul nou nu reflectă costurile de livrare și import;
- Reflectă cât de mult prețul poate fi ridicat. Așa numit „vibe-pricing”;

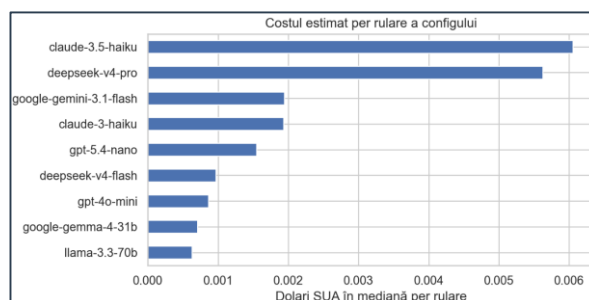
El este cerut să noteze 5 parametri în intervalul [-5; +5];

În general LLM-urile preferă să ridice prețul;
Cel puțin, poate depinde de promptul folosit;

Prețurile de rulare

Configurație	Identificatorul modelului	Prețul- intrare \$/1M	Prețul- ieșire \$/1M
claude-3-haiku	anthropic/claude-3-haiku	\$0,25	\$1,25
claude-3.5-haiku	anthropic/claude-3.5-haiku	\$1,00	\$5,00
deepseek-v4-flash	deepseek/deepseek-v4-flash	\$0,10	\$0,40
deepseek-v4-pro	deepseek/deepseek-v4-pro	\$0,27	\$1,10
google-gemini-3.1-flash	google/gemini-3.1-flash-lite	\$0,10	\$0,40
google-gemma-4-31b	google/gemma-4-31b-it	\$0,10	\$0,30
gpt-4o-mini	openai/gpt-4o-mini	\$0,15	\$0,60
gpt-5.4-nano	openai/gpt-5.4-nano	\$0,05	\$0,40
llama-3.3-70b	meta-llama/llama-3.3-70b-instruct	\$0,13	\$0,39

Prețul per 1M token-uri



Costul per rulare

Graficul din stânga:

- Chiar dacă costul per-token al Deepseek este mic, numărul de tokeni de ieșire este foarte ridicat.

Concluzii

- **Obiectiv:** nu un model câștigător, ci un *harness* care face LLM-urile măsurabile;
- Două rezultate, de naturi diferite:
 - Pozitiv-nuanțat: rezultatele descriu un LLM sub un anumit amestec de furnizori (rutarea OpenRouter), nu în absolut;
 - Aproape nul: disciplina JSON e prea uniformă (2 reîncercări, 0 eșecuri moale / 189 rulări) pentru a departaja LLM-urile;
- **Limitări:** rutare furnizori, 3 repetări (tendință, nu variantă), 7 anunțuri / o platformă, acuratețea agentului **pricer** imposibil de evaluat;
- **Directii viitoare:** modele locale (axa cloud-vs-local), variația promptului, a doua platformă (Shopify), intrări mai dificile, evaluare reală a prețurilor.